

RaccoonBot × OpenVLA 기반 mujoco Vision-Language-Action 파이프라인

데이터셋 확장, 코드 개선, 그리고 실물 RaccoonBot 제어

ga111o

2026년 6월 14일

요약

본 보고서는 mujoco 시뮬레이션 환경에서 OpenVLA-7B를 통해 4 DOF manipulator RaccoonBot 로봇을 제어하도록 하는 파이프라인 제시한다. 우선, 정육면체 객체 기반의 Pick-and-Place 태스크와 이에 대응하는 새로운 자연어 명령을 추가하고, 이를 기존 grasp 데이터와 통합해 단일 LoRA 기반의 멀티태스크 학습을 수행하였다. 이와 함께 4 DOF manipulator 로봇에 매핑하는 과정을 데이터 수집부터 실물 제어 전 단계에서 공통으로 사용하도록 일원화했다. 시스템 전반에 걸쳐 속도 및 메모리 최적화를 적용하고 진단 지표를 구축하여 observability를 확보하였다. 특히 단일 RTX 5080 (16GB) 환경에서 OpenVLA-7B의 학습과 추론을 동시에 수행할 수 있도록 asymmetric mixed-precision quantization을 구현하였으며, 구축된 추론 서버에 실물 로봇 클라이언트를 연동하여 실제 제어 검증까지 완료하였다. 결과적으로 비대칭 quantization을 통해 최대 학습 메모리를 약 14.1GB로 절감하여 16GB 단일 GPU에서의 학습을 성공적으로 구현했다. 약 5000 스텝의 학습 과정에서 과적합 없이 train loss는 약 0.27을 달성하였고, accuracy는 약 0.9가 나옴을 확인하였다. 하지만, 실제 mujoco closed-loop 평가에서는 시뮬레이션과 실물 모두에서 집기 성공률이 기대에 미치지 못하는 결과가 나왔다. 정성적 추측이 아닌 정량적 계측을 기반으로 closed-loop 제어 실패 원인을 규명하였다. 본 과제를 통해 Openvla를 통해 실제 7D-to-4DOF 액션 생성에 대한 파이프라인을 설계하였고, 실제 로봇에 적용하는 귀중한 경험을 할 수 있었다.

1 시스템 개요 및 아키텍처

1.1 대상

본 과제의 대상 로봇은 4 DOF manipulator와 gripper를 갖춘 RaccoonBot이다. mujoco 시뮬레이션 환경의 모델과 실물 RaccoonBot은 동일한 kinematics를 공유한다(6장). 정책 모델로는 OpenVLA-7B를 사용한다. 이 모델은 DINOv2와 SigLIP을 결합한 이중 비전 백본, Llama-2 7B 언어모델, 그리고 256구간 행동 토큰라이저로 구성되며, LoRA로 파인튜닝한다. 시뮬레이터는 헤드리스 소프트웨어 렌더링으로 동작하는 mujoco이고 관측은 카메라 RGB 영상이다. 실물 환경에서는 동일한 관측 인터페이스를 실제 카메라 프레임으로 대체한다.

1.2 전체 흐름

우선, mujoco를 활용해 pick-and-place task에 대한 데모를 병렬로 생성한다. 수집된 데이터는 로봇 train data 형식으로 변환 및 구축되며, 이를 바탕으로 OpenVLA-7B 모델에 대한 LoRA 파인튜닝을 수행한다. 학습이 완료된 모델은 시뮬레이션 평가와 실물 제어 환경 모두에 inference를 진행한다. 이 모든 과정은 단일 진입점(2장)을 통해 모듈별로 선택적 실행이 가능하며, 실행 내역은 모두 시스템 로그로 기록되도록 하였다.

모델이 7D action space [dx, dy, dz, droll, dpitch, dyaw, gripper] 상의 명령을 출력하면, 클라이언트는 사전에 정의된 공통 행동 매핑 로직(4.1절)을 적용하여 이를 4 DOF 로봇 제어를 위한 IK로 해석한다. 보상은 sparse reward로 진행하였다. 특히, 추론 서버는 요청을 발생시키는 클라이언트의 구체적인 작동 환경(시뮬레이션 또는 실물)에 의존하지 않도록 독립적으로 설계되었다. 이러한 서버와 클라이언트 간의 디커플링(Decoupling) 구조는 시뮬레이션 환경에서 검증된 정책을 실물 로봇으로 이식(Sim-to-Real Transfer)할 때 시스템의 일관성과 신뢰성을 보장하는 핵심적 근거가 된다(5장, 6장).

2 파이프라인 인프라

2.1 설계 개요

각 실행 단계를 독립적인 스크립트로 분리하되 번호를 부여하여 관리할 수 있도록 하였다. 이때, OpenVLA 코드베이스 전체를 수정하는 것은 한계가 있기에 대부분은 기존 코드를 하위 subprocess로 호출하는 wrapper 형태로 작성했다. 이때, 전체 파이프라인은 단일 진입점으로 제어되도록 하였다. 또한, 분산 학습이나 mujoco 환경에서 zombie 혹은 orphan process를 방지하기 위해, 정상 종료나 시그널 발생 시 파생된 모든 하위 프로세스까지 깔끔하게 정리하도록 하였다.

2.2 argument 설정 및 실행 기록

각 단계의 진행 여부와 argument는 config 파일에 작성하였으며, 적용 우선순위는 명령행 인자, 설정 파일, 코드 기본값 순이다.

매 실행마다 Git 커밋 해시, 단계별 종료 코드 및 소요 시간, 선택된 단계, 그리고 최종 상태를 JSON manifest 형태로 기록하도록 하였다. 실행이 실패하거나 중간에 중단되더라도 해당 시점까지의 부분 기록이 보존되며, 콘솔 창에는 단계별 소요 시간이 표시된다. 파이프라인을 구성하는 세부 단계는 표 1과 같다.

표 1. 단계형 파이프라인 구성

단계	동작	GPU
환경 점검	경로, 라이브러리, 패키지 등 확인	—
데모 생성	집기·쌓기 데모 생성	(CPU 병렬처리)
데이터 변환	raw 영상 데이터를 변환 후 학습 데이터셋 빌드	—
에피소드 시각화	프레임 몽타주 이미지 생성	—
학습	샌드위치 QLoRA	사용
병합·추론	LoRA 사후 병합 및 FP8 추론 서버 구동	사용
오프라인 검증	예측값과 정답 간의 거리 비교 분석	사용
closed-loop 평가	시뮬레이션 환경 집기·쌓기 성공률 검증	사용
실제 제어	추론 서버와 클라이언트를 통한 RaccoonBot 구동 (6장)	사용

3 데이터셋 확장

3.1 확장 내용

기존 파이프라인은 대상 객체가 원기둥에 국한되고, 수행 가능한 작업이 단순한 '집기'뿐이며, 명령어 역시 "원기둥을 집어라" 단 하나로 고정되어 있다는 한계가 존재했다. 본 과제에서는 정육면체

객체와 '집어서 쌓기' task를 도입하고 이에 대응하는 명령어를 추가하였다. 첫 과제 안내에서는 실제 로봇이 아닌, 시뮬레이션 환경에서만 동작만을 요구하였기에 RaccoonBot 세트에는 존재하지 않는 큐브를 정의하였다. 다만, 실제 동작에는 큰 영향을 끼치지 않는다고 판단하여 그대로 진행하였다.

3.2 쌓기 성공 판정

성공 판정은 기존 예제의 grasp용 접촉 기준을 그대로 쓰지 않고 새로 정의했다. 큐브의 평면 위치가 베이스 위치의 허용 오차 이내이고, 높이가 베이스 상단에 큐브 절반 높이를 더한 값에 허용 오차 이내로 맞으며, 큐브가 베이스와 접촉하고, 그리퍼가 열려 있으며, 움직임이 충분히 느려 안정된 상태를 성공으로 본다. 쌓기는 본질적으로 해제 후의 안착이 중요하기에, 접촉만으로 판정하면 적절하지 않기 때문이다.

4 코드 개선

4.1 7D-to-4DOF

7D-to-4DOF RaccoonBot에 대응시키는 매핑 로직을 하나로 통합하여, 데이터 수집, 학습 데이터 생성, closed-loop 시뮬레이션 평가, 그리고 RaccoonBot 제어 등 모든 단계에서 동일하게 재사용할 수 있도록 구조를 개선하였다.

또한, step별 위치 변위의 axis별 제한값을 argument로 설정할 수 있도록 하여, 학습 데이터 분포의 상위 분위수에 맞춰 유연하게 조정할 수 있게 했다. 매핑 함수는 초기 변위, 최종 적용 변위, 목표 위치, gripper 명령, IK 재시도, 실패 여부 등 다양한 진단 정보를 반환하며, 평가 단계에서는 이를 로깅 및 분석에 활용한다(4.3절).

특히, end-effector의 자세를 도출하는 kinematics 연산은 grasp task에서 큰 오차가 발생함을 확인하여 데이터 수집 단계와 시뮬레이션 환경 단계가 동일한 kinematics 상수를 공유하여 수행되도록 통일했다. RaccoonBot 제어 또한 동일한 매핑을 사용하여, 시뮬레이션에서 검증된 동작을 실제 로봇이 재현할 수 있도록 하였다(6장).

4.2 추론·모션 속도 및 메모리 최적화

학습, 추론, 모션 제어, 데이터 생성에 이르는 파이프라인 전 구간에 걸쳐 연산 속도 향상 및 메모리 최적화 기법을 도입했다. 각 구간별로 적용된 구체적인 기법과 그에 따른 성능 개선 효과를 표 2에 요약했다.

개선 전후 비교는 학습 로그의 최대 할당·예약 메모리와 스텝 시간으로 정량화, inference latency는 평가 리포트의 평균 inference latency와 실물 제어 시 서버 로그의 타임스탬프 간격으로 측정한다(6장).

4.3 타이밍 및 행동 로깅과 시각화

본 파이프라인은 실험의 재현성을 보장하고 제어 실패 원인을 정밀하게 분석하기 위해 로깅 및 시각화 체계를 구축하여 시스템 observability을 확보하였다. 우선, 시스템 실행 기록(2.2절)을 통해 Git 커밋 해시, 모듈 단계별 종료 코드 및 소요 시간을 자동으로 기록하여 실험 간 성능 비교를 위한 기반 데이터를 제공한다. 이와 더불어 Closed-loop 평가 과정에서는 매 스텝마다 clipping 발생률, gripper 폐쇄 유지 스텝 수 IK 해 탐색 실패 및 재시도 횟수, end-effector의 누적 이동 거리 등의 세부 제어 지표를 수집하여 평가 리포트로 산출한다. 이러한 정량적 계측 데이터는 제어 실패 원인이

표 2. 구간별 최적화 기법 및 효과

기법	구간	효과
Flash-Attention 2	학습	attention buffer를 제거하여 활성화 메모리를 줄이고, kernel fusion으로 연산 속도 향상.
메모리 fragmentation 완화 할당자	학습	메모리 fragmentation로 인해 낭비되는 메모리 공간을 최소화.
8bit optimization	학습	LoRA optimization state를 32bit에서 8bit으로 quantization.
LoRA 사후 병합	학습	매 체크포인트마다 14GB 규모의 weight를 병합하는 대신, 학습 종료 후 1회만 병합, 전체 throughput을 극대화
물리 연산 반복 축소	데모 생성	physics pass 횟수를 100회에서 50회로 축소하여 step당 연산 비용을 절감. 50까지 줄였음에도 anti-slip에 문제가 없음을 확인하였다.
병렬 롤아웃 워커	평가	멀티코어 기반으로 병렬로 rollout을 수행하며, 모든 worker가 단일 GPU 기반 policy server를 통해 평가 효율 향상.
FP8 추론	추론	Blackwell tensor core의 FP8 가속 및 deterministic decoding 활용 RaccoonBot 제어 주기 latency 최소화

부적절한 action scale 출력에 있는지, gripper grasp 타이밍의 문제인지, 혹은 단순 IK 연산 한계인지를 객관적으로 판별하는 데 활용된다(??장).

또한, 결과 분석의 직관성을 높이기 위해, 학습 데이터 검증에 위한 에피소드 프레임 몽타주, 평가 지표 집계 그래프, 추론 과정에서 로봇의 동작을 프레임 단위로 보여주는 롤아웃 프레임 몽타주를 생성한다. 이에 더해, ClearML을 바탕으로 실험 추적을 진행하여 train loss부터, accuracy, L1 loss 등을 체계적으로 관리할 수 있도록 하였다.

5 추론 및 평가

오프라인 검증은 데모 프레임 한 장에 대한 예측과 정답의 거리만을 확인한다. 학습이 실제로 이루어졌는지를 사람이 눈으로 가늠하는 정도이다. 다만, 이로는 실제 로봇을 작동시킬 때, 오차가 크다는 것을 확인하여 정책을 mujoco에서 여러 차례 롤아웃하여 실제 과제 성공률(수집과 동일한 성공 기준을 재사용), 성공까지의 스텝 수, 평균 추론 지연을 자동으로 산출하는 closed-loop 평가를 진행하였다. 베이스 모델과 파인튜닝 모델의 성공률 차이로 학습 효과를 정량화하였다.

서버는 명령 텍스트와 카메라 이미지를 받아 7차원 행동을 돌려주는, state가 없는 HTTP 예측 엔드포인트일 뿐이며, 누가 요청하든 동일하게 응답한다.

추론과 mujoco 소프트웨어 렌더링을 한 프로세스에 올리면 충돌이 발생함을 확인하였으며, 완전히 뜯어고치지 않는 한 구조적으로 해결이 불가능함을 확인하였다. 렌더링 worker를 별도 프로세스로 분리하고, wrapper가 서버 동작부터 워커 실행, 서버 종료를 순서대로 진행한다. 이는 충돌 회피를 넘어 decoupling도 된다. 같은 서버에 시뮬레이션 평가 워커 대신 실물 클라이언트를 연결하면 곧바로 실물 제어도 가능하다(6장).

6 RaccoonBot 제어

RaccoonBot 제어는 RaccoonBot이 RaccoonBot 및 카메라와 연결된 클라이언트가 추론 서버와 통신하는 방식으로 하여 RaccoonBot을 구동하였다.

6.1 배포 구조

수업 때 받은 GPU 서버는 포트포워딩 불가 문제로 외부에서 직접 접근할 수 없어, FP8 추론 서버는 개인 데스크탑에 띄웠다. 노트북은 실물 RaccoonBot과 카메라와 직접 연결하여, 노트북에 물린 카메라의 프레임을 캡처하고, 서버에 예측을 요청해서 7D action을 받은 뒤, 공유 행동 매핑으로 4DOF와 IK를 풀어 RaccoonBot SDK를 통해 모터에 명령을 내린다. 서버는 본 과제에서 개선한 FP8 추론 서버로, 7B 정책을 단일 16GB VRAM에 올리고(7장) latency를 최소화 하여 real-time 제어를 가능하게 하였다.

6.2 시뮬레이션 검증의 실물 이식성

본 연구에서는 데이터 수집, 학습 라벨링, 시뮬레이션 평가, 실물 적용이라는 네 단계 전반에 걸쳐 동일한 7D-to-4DOF(4.1절의 동일한 축별 클립, 동일한 IK, 동일한 kinematics 상수)를 적용하였다. 이를 closed-loop 시뮬레이션 환경에서 측정한 성공률과 실패 양상이 실제 구동 시의 신뢰성을 예상할 수 있도록 하였다.

6.3 실물 결과(증거)

실물 RaccoonBot은 시뮬레이션 집기 정밀도(??절)가 충분히 오른 체크포인트로 구동하며, 측정 수집 항목은 다음과 같다.

- [실물 데모 영상]: 대상 색별(빨강·파랑·초록·노랑) 도달과 집기, 그리고 쌓기 1회 이상.
- [서버 예측 로그와 실물 모션 타임스탬프 정합 캡처]: 추론 출력과 실물 동작이 일치함을 증명.
- [실물 성공·실패 표]: 색별 시도·성공과 주요 실패 모드(집기 정밀도, 시뮬레이션-실물 간극 등).
- [제어 주기 지연]: FP8 서버의 평균 응답과 실물 한 스텝 소요.

7 VRAM 절감과 비대칭 Quantization

수업에서 접속권한을 받은 A100 서버의 가용 자원이 일관되지 않아 개인 데스크탑(RTX 5080(Blackwell, 16GB))으로 학습 및 추론을 진행하였다. 이때, OpenVLA-7B의 BF16 전체 LoRA 학습은 16GB에서 메모리 부족으로 실패하였다. 이를 해결하기 위해 모듈별 비대칭 mixed-precision 샌드위치 구조의 QLoRA를 구현했다.

7.1 모듈별 비대칭 Quantization

VLA는 모듈마다 quantization 민감도가 크게 다르기에, 모든 모듈에 같은 quantization을 적용하는 방식은 피해야 한다. quantization은 한 곳으로 모아 학습과 추론이 공유한다. 모듈별 학습 및 추론 precision은 표 3와 같다.

OpenVLA는 수업에서 배웠던 것과 같이 로봇 제어를 위해 독립적인 action head 모듈을 두지 않고, 언어 모델의 최종 출력 logits 중 256개의 action vocabulary에서 최댓값을 얻고, 이를 denormalization해서 연속적인 제어 값으로 변환하는 방식을 사용한다. 만약, 시스템 경량화를 위해 언어 모델의 백본 전체를 FP8로 Quantization할 경우 최종 출력 logit의 미세한 오차가 누적되어 제어에 실패할 위험이 있다. 따라서 본 프로젝트에서는 양자화 범위를 언어 모델 본체로만 제한하고, action logit을 연산하는 출력 층은 quantization하지 않도록 하였다.

표 3. Asymmetric Mixed-Precision

모듈	학습	추론
ViT(DINOv2 + SigLIP)	BF16 + LoRA ($r = 16$)	INT8 Post-training Quantization
프로젝터(MLP)	BF16 전체 미세조정	BF16 유지
LLM backbone(Llama-2 7B)	FP8 freeze + LoRA ($r = 32$)	FP8 (W8A8)
Action Output Layer	BF16 전체 fine-tune	BF16 유지

7.2 구현

OpenVLA 아키텍처가 특정 transformers 버전을 요구하기에 모델 가중치를 우선 메모리에 올리고, 이후 독립적인 모듈로 quantization을 하였다. 샌드위치 QLoRA 구성에서는 언어 모델 백본을 FP8로 quantization 및 freeze(약 7.5GB)한 상태에서 어텐션의 Q, V projection층에 LoRA($r = 32$)를 부착하였고, vision encoder는 BF16에서 LoRA($r = 16$)를 적용하였다. vision-language projector와 action head는 full fine-tuning 대상으로 하여 BF16으로 학습을 진행하였다. 학습 효율성을 높이기 위해 gradient checkpointing, 8-bit optimizer, flash-attention 2으로, batch size 4와 gradient accumulation steps 4를 설정하여 유효 배치 크기 16을 달성할 수 있었다.

추론 단계에서는 현재 vLLM 프레임워크가 prismatic 기반 이중 백본을 지원하지 않아, 기존 FastAPI 기반의 추론 서버를 구현하였다. 특히 OOM을 방지하기 위해, CPU 메모리에 모델을 먼저 로드한 후 layer-wise하게 quantization하여 GPU로 넘기도록 하였다. 이를 통해 약 14.5GB의 VRAM만으로 모델을 올릴 수 있었다.